Calculation Walkthroughs — How Every Result-Panel Number Is Computed

One markdown file per test scenario in the parent folder. Load the scenario, calculate, then read the matching file side-by-side with the result panel: each section of the file mirrors a section of the panel and shows the arithmetic behind the numbers on screen.

> **New here? Read 00-ORIENTATION first.**
>
> This file is a **formula sheet** — it assumes you already know what a shaft generator is and why a coverage factor exists. `00-ORIENTATION.md` explains the vocabulary, the reasoning behind each step, the three demand "worlds" that are easy to confuse, and a **reverse index**: "*I see this number on the screen — which step produced it, and which scenario shows it most clearly?*"

All figures were verified against a live API instance built from commit `559aef2` (2026-07-22). Fuel price is **$780/ton** unless the file says otherwise — $ figures scale linearly with price.

The arithmetic below is still current. The backend has been refactored since that commit, but every step was proven behaviour-preserving against 18 golden snapshots compared byte-for-byte, and the client was refactored without touching the request it sends. See `docs/refactoring/backend-refactor-design.md` and `docs/refactoring/client-refactor-design.md`.

The 7-step recipe (used by every walkthrough)

Step 1 — Battery cascade (Excel "Load Demands" sheet). The battery power budget is handed out over the loads in strict priority order: DpReserve → DpDemand → Mission → Propulsion → Hotel. Per row:

```
H = what the row wants     PEAK SHAVING rows: avg load × variation factor (propulsion 5%, hotel 2%)
                           MISSION row:       the full heavy-consumer max (it can start any moment)
                           RESERVE row:       the full requirement (class rule, kW for kW)
I = min(remaining budget, H)          what the row takes from the budget
J = I × coverage factor               what actually counts as covered
                                      (RESERVE 100% · DP/Mission 50% · propulsion 35% · hotel 5%)
L = H − J                             what the gensets must still carry (spinning reserve)
remaining −= I                        the next row sees a smaller budget
```

Tiles: **Peak Shaving = ΣJ** (peak-shaving rows) · **Spinning Reserve = ΣL**. Invariant: ΣJ + ΣL = ΣH — the sea sets the total swing; the battery only moves the split.

Step 2 — Adjusted demands. Per side: `propulsion' = propulsion_avg + L(propulsion-side rows)`, `hotel' = hotel_avg + L(hotel-side rows: hotel, mission, DP-hotel)`. Covered J never reduces demand.

Step 3 — Load distribution (per candidate combination ME-count × SG × AE-count):

```
SG power = min(hotel', SG capacity of ACTIVE MEs)     SG covers hotel first
AE power = min(hotel' − SG power, AE capacity)        AE takes the remainder
ME power = propulsion' + SG power                     the SG is a shaft load ON the ME
load %   = power / capacity of the ACTIVE units
```

Step 4 — SFOC → FOC. `FOC [t/h] = P_ME × SFOC_ME(load%)/1e6 + P_AE × SFOC_AE(load%)/1e6`. SFOC is linearly interpolated from the selected engine type's curve (DB), extrapolated below the lowest point.

Step 5 — Ranking. Sort valid combos by FOC. **Optimal = first row** (this IS Integration Level 1 — it is not user-selectable). **Baseline** = last row (no battery) or **3rd-from-worst** (battery active, clamped for short lists); the radio in "Assumed Configuration" can override it.

Step 6 — Annual numbers. `FOC t/yr = t/h × mode hours` (summed over modes) · `CO2 = FOC × per-fuel factor` (MGO/MDO 3.93267 · HFO 3.114 · LNG 2.753) — ME and AE each use their own fuel's factor · `cost = FOC × fuel price` · `iEMS savings = baseline − optimal` · tier investments are fixed constants ($110k / $140.5k / $210k).

Step 7 — Battery Benefit (the green badge; dual-scenario rule R3a). The code runs Level 1 twice: world A (with battery, demand = avg + ΣL) and world B (reference: budget 0, demand = avg + ΣH). `Benefit = max(0, FOC_B_optimal − FOC_A_optimal) × hours`, summed per mode; `$ = benefit × fuel price`. The baseline radio never touches this number.

> **Do not try to reproduce a Benefit by unticking "Enable Battery" in the UI.** That produces a *third* world — raw demand with the swing carried by nobody at all — which burns **less** than the battery case and makes the benefit look negative. World B is scenario **03**: the same ship with the full swing written into the loads. `00-ORIENTATION.md` Part 4 has the three side by side.

Files

`00-ORIENTATION.md` — start here: vocabulary, the reasoning behind each step, the three worlds, and the reverse index.

`00a-PIPELINE-DIAGRAM.html` — the same machine drawn, with scenario 01's numbers flowing through it. Three figures: how the cascade splits a constant, how demand becomes fuel, and why the Battery Benefit needs two runs of the pipeline. Read it beside `00-ORIENTATION`.

01–12 mirror the first-wave scenarios, 13–18 the second wave, 19–35 the third (see below). Error scenarios (09, 17) explain the rejection math instead of panels; 18 explains the deliberate absence of the battery panel.

A reading order that covers every mechanism without reading all 35: `01 → 02 → 03 → 04 → 19 → 11 → 05 + 06 → 09 → 12`. Full reasoning in `00-ORIENTATION.md` Part 7.

PDF versions

`pdf/` holds a rendering of every file here, plus the two documents one level up that are needed to actually run the tests — the scenario list with its expected results (`SCENARIOS-AND-EXPECTED-RESULTS.pdf`) and `COVERAGE-MATRIX.pdf`.

Regenerate after any edit:

```
node build-pdf.cjs
```

No dependencies — a small Markdown renderer plus headless Chrome, found automatically (override with `CHROME_BIN`). Links between documents are rewritten to point at the `.pdf`.

The set used to be produced by hand. It drifted: the last manual render covered 01–18 only, missed the 2026-07-28 revision of card 04, and mangled the cascade tables — cells landed on the wrong rows, which is exactly the content these documents exist for. A script beats a habit.

Scenarios 19–35 (added 2026-08-04)

The suite originally grew feature-by-feature, which left it deep on battery behaviour and thin elsewhere. Reading the **snapshots** rather than the scenario files exposed three blind spots:

- **Level 2 produced zero savings in all 18** scenarios — a whole optimization level with no end-to-end coverage.
- **PTI assist never engaged**, despite two scenarios named after it.
- **Only 2 of the 4 Level 1 rejection messages** a user can receive were reachable.

Scenarios 19–35 close those and the smaller gaps listed in `../COVERAGE-MATRIX.md`.

Read their cards differently from 01–18. The earlier figures were derived from the reference workbook and then compared with the code. The newer ones came *from* the code; where the arithmetic could be reproduced by hand the card shows it under **Hand-check**, and where it could not, the card says **pending reference verification**. They are reliable change detectors today and become correctness proofs once the workbook confirms them.
